

Enterprise Transport API

C# Edition

3.5.1.L1

DACSLOCK API DEVELOPERS GUIDE



© LSEG 2024 - 2026. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any software, including but not limited to: the code, screen, structure, sequence, and organization thereof, and its documentation are protected by national copyright laws and international treaty provisions. This manual is subject to U.S. and other national export regulations.

LSEG Data & Analytics, by publishing this document, does not guarantee that any information contained herein is and will remain accurate or that use of the information will ensure correct and faultless operation of the relevant service or equipment. LSEG Data & Analytics, its agents, and its employees, shall not be held liable to or through any user for any loss or damage whatsoever resulting from reliance on the information contained herein.

Contents

1	Introduction	1
1.1	Product Description	1
1.2	IDN Versus LSEG Real-Time	1
1.3	Audience	1
1.4	Organization of Manual	2
1.5	References	2
1.6	Conventions	2
1.6.1	<i>Typographic</i>	2
1.6.2	<i>Programming</i>	2
1.7	Glossary	3
2	Requirements for Compliant Source Server Applications	5
2.1	Overview	5
2.2	Establish Subservice Names	5
2.3	Define Entitlement Codes	5
2.4	Create and Write Locks	5
2.5	Publish the Map	6
2.6	Read the Map and Supply it to the Data Access Control System Station	6
3	DACSLock API	7
3.1	DACSLock Operation	7
3.2	Forming a DACSLock	8
3.3	DACSLock Contents	9
3.4	Compression of DACSLocks	10
3.5	Compounding DACSLocks	11
3.6	Transport of DACSLocks	11
3.7	Compound DACSLock in Relation to Permissioning	12
4	DACSLock API Interface	13
4.1	AuthorizationLock Interface	13
4.1.1	<i>AuthorizationLock Enumeration</i>	13
4.1.2	<i>AuthorizationLock Construction</i>	13
4.1.3	<i>AuthorizationLock Member Functions / Properties</i>	15
4.2	AuthorizationLockData Interface	16
4.2.1	<i>AuthorizationLockData Enumeration</i>	17
4.2.2	<i>AuthorizationLockData Construction</i>	17
4.2.3	<i>AuthorizationLockData Member Functions / Properties</i>	18
4.3	AuthorizationLockStatus Interface	20
4.3.1	<i>AuthorizationLockStatus Construction</i>	20
4.3.2	<i>AuthorizationLockStatus Member Property</i>	21
	Appendix A Example Program	22

1 Introduction

1.1 Product Description

The goal of permissioning is to control access to data by users. Using an entitlement system such as the Data Access Control System, permission profiles can be defined identifying what each user is allowed to access. The Data Access Control System is the entitlement system for LSEG Data Management Solutions (LSEG Real-Time Distribution System). Data Access Control System permission checks take place in the Market Data Client application. In order to perform a permission check for an item, the Data Access Control System must have 'requirements' information for the item and a profile for the user (a.k.a. content based permissioning).

Data Access Control System requirements information is organized as numeric expressions. Each subservice of a service, e.g. all the data from an exchange, is assigned a (series of) numeric **entitlement codes (PEs)**. The numeric entitlement codes are transported with items as they move from source servers to Market Data Client applications on the LSEG Real-Time Distribution System. In order to accommodate the most general case in which information from multiple sources is combined to form new items (such as in compound servers), the requirements information for an item is a Boolean expression containing these entitlement codes. For an item obtained directly from a source, this expression is usually a single term. When a compound server combines two items to form a new (compound) item, it must also combine the requirements information to form a new (compound) requirement.

The name of the subservice associated with an entitlement code is maintained in tables within the Data Access Control System database and operational permission checking subsystem. The table that relates the entitlement codes for a service to the subservice names for that service is called the **map** for the service.

Requirements are transported on LSEG Real-Time Distribution System in protocol messages called locks. The DACSLock API provides functions to manipulate locks in a manner such that the source application need not know any of the details of the encoding scheme or message structure. For a source server to be Data Access Control System compliant, based on content, it must publish locks for the items it publishes; i.e., the source server application must produce lock events. Any item published without a lock or with a null lock is available to everybody permitted for that service, even those without subservice permissions.

If a source server introduces (new) data to LSEG Real-Time Distribution System that originates outside the network on which the application is running, then the application developer is also responsible for providing the map information for the service.

In addition to source servers that publish new data directly from a vendor, there are also compound servers that gather market information from other sources, manipulate that information, and then re-publish it on LSEG Real-Time Distribution System. These servers must read locks from the Transport API C# Edition to combine them before publishing compound items. Again, the DACSLock API provides functions to assist with this process. Compound sources must also use a special form of user ID when connecting to the LSEG Real-Time Distribution System network.

NOTE: Subject-based sources do not require locks.

1.2 IDN Versus LSEG Real-Time

LSEG's new ultra-high speed network LSEG Real-Time has replaced the older IDN network. All references herein are made to LSEG Real-Time. However, for historical reasons in the Data Access Control System administrative screens, this network is still referred to as IDN. For this reason, the terms LSEG Real-Time and IDN are interchangeable throughout this document.

1.3 Audience

This guide is intended for software programmers who wish to incorporate the DACSLocks into the development of their source applications.

1.4 Organization of Manual

The material presented in this guide is divided into the following sections:

CHAPTER	CONTENT / TOPIC
Chapter 1, Introduction	A description of this manual and its conventions.
Chapter 2, Requirements for Compliant Source Server Applications	For subservice-level permissioning, a DACS-compliant source server must satisfy a series of requirements.
Chapter 3, DACSLock API	Explains the data flow of a DACSLock and its operations.
Chapter 5, DACSLock API Components	Describes DACSLock API components and their required properties.
Appendix A, Example Program	Lists an example program that was created using the DACSLock API.

Table 1: Manual Overview

1.5 References

- *Transport API C# Edition Developers Guide*
- *DACSLock API Reference Manual*

1.6 Conventions

1.6.1 Typographic

This manual observes the following typographic conventions:

- C# classes, methods, and types are shown in **Courier New** font.
- Parameters, filenames, tools, utilities, and directories are shown in **Bold** font.
- Document titles and variable values are shown in *italics*.
- When initially introduced, concepts are shown in ***Bold, Italics***.
- Longer code examples (one or more lines of code) are shown in Courier New font against an orange background. Code comments are shown in green font color. For example:

```
AuthorizationLock authLock = new (5000, AuthorizationLock.OperatorEnum.OR, new List<uint>{63});
authLock.AppendPE(62);
AuthorizationLockData lockData = new();
AuthorizationLockStatus retStatus = new();
LockResultEnum result = authLock.GetLock(lockData, retStatus);
```

1.6.2 Programming

Enterprise Transport API C# Edition Standard conventions were followed.

1.7 Glossary

TERM	DESCRIPTION
API	Application Programming Interface
Application	A program that accesses data from and/or publishes data to the system.
Compound Item	A data item prepared from data items retrieved from the system.
Concrete service	A set of real-time data items published by a source server. Each concrete service is identified on the Data Access Control System by a unique name (known as a network).
Data Access Control System	An entitlement tool that allows customers to automatically control who is permitted to use which sets of data in their LSEG Real-Time Distribution System deployment.
Entitlement Code	If a vendor service is permissioned down to the subservice level, an entitlement code must be provided with each item. Based on mapping tables provided by the vendor, the Data Access Control System uses this code to determine the information provider and/or vendor product associated with an item.
Exchange	A commercial establishment at which or through which trading of financial instruments takes place. Exchanges are information providers.
Exchange Map Logic	The logic used to construct requirements when an item is supplied by more than one exchange. If OR logic is used, the user only needs permission to access one of the exchanges that supplies the item. If AND logic is used, the user must have permission to access all of the exchanges that supply the item.
Map Program	An application associated with a particular vendor service which requests permissioning data from the vendor host and then uses that data to construct various mapping tables required by the Data Access Control System.
Mapping Tables	These tables are used by the Data Access Control System to derive the requirement for an item. They map entitlement codes to vendor products and, if applicable, to information providers (exchanges and specialist services).
Network Service	See concrete service.
PE	Permissionable Entity. A number used to designate the permissioning basis of a data item on LSEG Real-Time (or LSEG Real-Time Distribution System). (Same as entitlement code.)
Permissioning	The control of access to and publication of data items by users.
Product	A subset of the data items delivered by an information vendor for which there is a single charge (based on vendor criteria).
Product Map Logic	The logic used to construct requirements when an item is supplied by more than one product. If OR logic is used, the user only needs permission to access one of the products that supplies the item. If AND logic is used, the user must have permission to access all of the products that supply the item.
Profile	Information, including a list of subservices, that is used during permission checking. There is a profile associated with each user.
LSEG Real-Time	LSEG's open, global, ultra-high-speed network and hosting environment, which allows users to access and share various types of content.
RDF Direct	Data Feed Direct
Service	This term is used in two ways by the Data Access Control System in a market data system environment. See concrete service and vendor service.
Service ID	A unique, numeric ID assigned to each network service. On an LSEG Real-Time Distribution System network, all valid service IDs for a particular system are listed in the global configuration file <code>rm.ds.cnf</code> .

Table 2: Glossary and Acronyms

TERM	DESCRIPTION
Source	An application or server capable of supplying or transmitting information.
Source Server	An application program which provides a concrete service. More than one source server can provide the same concrete service.
Specialist Data Service	A set of data items provided by a third party (i.e. not from an exchange and not from the vendor delivering the data items to the site).
Subservice	A named set of items delivered by a vendor which are authorized as a group; e.g. all instruments traded on NYSE or all items that make up the product Securities 2000.
Subservice Type	There are three types of subservices: • Products • Exchanges • Specialist Service
User	A person with a unique, system-wide name.
Vendor Service	A vendor may offer more than one type of data delivery service to a customer. For example, LSEG provides the LSEG Real-Time service. Each vendor service is identified on the Data Access Control System by a unique name. Each vendor service is associated with one or more concrete services. For example the LSEG Real-Time service may be published on the network by any combination of these concrete services: IDN Selectserver and Reuters Data Feed.

Table 2: Glossary and Acronyms (Continued)

2 Requirements for Compliant Source Server Applications

2.1 Overview

To permission at the subservice level, the item's permissioning requirements must be available at locations other than just the source. For the Data Access Control System to permission a source service below the service level, the source must:

1. Create locks containing permissioning information, and
2. Map entitlement codes in the locks to subservice names.

For subservice level permissioning, a DACS-compliant source server must:

- Establish Subservice Names
- Define Entitlement Codes
- Create and Write Locks
- Publish the Map
- Read the Map and Supply it to the Data Access Control System Station

2.2 Establish Subservice Names

If a source server provides a service that is subdivided into subservices, each subservice must have a symbolic name. For Real-Time, symbolic names represent LSEG products, exchanges, or specialist data services, and are subsets of the service being provided by the vendor. An item might be in zero, one, or more subservices.

At the time a source publishes an item on the LSEG Real-Time Distribution System, the source must associate with the item the identities of any subservices to which the item belongs. For example, a Real-Time source might identify that an item belongs to the subset of items from the New York Stock Exchange and is part of the Equities 2000 product.

For users, the system administrator grants or denies access to items in the various subservices using symbolic names. The system administrator performing permissioning will not deal with arbitrary numeric encodings such as via PEs (Permissionable Entity).

2.3 Define Entitlement Codes

The second requirement is that an entitlement code (a number) must be associated with each subservice name. A subservice can have one or more associated entitlement codes. Whenever a source publishes an item, the source designates the subservice(s) to which the item belongs as a Boolean expression in entitlement codes. Entitlement codes are needed when combining permissioning information from multiple sources to permission compound items (i.e., made from more than one source).

The PE used for permissioning on IDN (FID 1, PROD_PERM) is an example of an entitlement code.

The list of subservice names and associated entitlement codes is called the *map* for the source.

2.4 Create and Write Locks

Whenever a source server opens a data stream, it must write a lock containing the item's entitlement code expression. The source server must use an existing Real-Time API to publish permissionable data on the LSEG Real-Time Distribution System.

One of the capabilities of such an API is to create the lock. The arguments to the API function include a list of entitlement codes and the Boolean operator (AND or OR), which indicates how to logically combine them. After the lock is created, it must be posted to the LSEG Real-Time Distribution System.

A source application can post a revised lock at any time.

2.5 Publish the Map

The source must publish, as data items, the map of its entitlement codes and subservice names (or use **map_generic** to load a map through the use of a file). As an example, the map for the Real-Time service is published as a series of RICs referred to as the Reuters Product Definition Pages.¹

Within the map data there must be a readily available data item with a date-time stamp. The value of this date-time stamp must be the date and time at which the map was last changed. The objective is to permit the application that reads the map to read a single item and determine whether the remaining data has changed (and thus needs to be reread).

The map items (records or pages) must be available to a user and application that have no subservice permissions so that the map can be retrieved at a new site that does not yet have permissions distributed.

NOTE: For Real-Time services, template files containing preliminary mapping information are provided with the Data Access Control System software so that subservice permissioning can be set up before the latest map is retrieved from the source. If a template file is not provided with a third-party source application, it is important to not require subservice permissioning so that the map can be retrieved from the source.

2.6 Read the Map and Supply it to the Data Access Control System Station

There are two ways to load map data:

- Use the Generic map collect program (**map_generic**).
- Download the map.

The Data Access Control System database must contain the source's map data so that:

- The Data Access Control System administrator can assign permissions to subservices for a service
- The Data Access Control System operational subsystem can perform permission checks

As mentioned previously, the source should publish this map. Additionally, the source application developer should provide a map collection program designed to:

- Request the map items from the source.
- Determine if there were any changes since the last map was received.
- Convert any revised information into a file that can be loaded into the Data Access Control System database.

The source application developer may also want to provide a map monitor program that can run periodically to retrieve the latest map and see if any changes have occurred since the last map collection (by checking the date-time stamp). Based on the status reported by the monitor program, the administrator knows when the map collection program needs to be run.

The Data Access Control System does not care about the format of the map published by the source as long as the map collection program for that source produces an appropriately formatted permission map file.

For further details on the map collect and proper permission map file formatting, refer to the *DACSLock API Reference Manual* specific to the version of the Data Access Control System that you run.



TIP: The Data Access Control System software package comes with a map collection program for the Real-Time service.

1. Refer to the *Reuters Product Definition Pages User Guide* to see how LSEG communicates permissioning information for its products.

3 DACSLock API

3.1 DACSLock Operation

The DACSLock contains requirements for an item that a vendor source deems necessary. On the Market Data Client LAN, users are entitled to specific capabilities. Therefore, when a Lock, which contains requirements, is tested against the capabilities of the user, enough information is available to permission the following:

PERMISSIONING OPERATION
USER -> APPLICATION
USER -> SERVICE
USER -> SUB-SERVICE (entitlement codes)

Table 3: Data Access Control System Permissioning Capabilities

DACSLocks are critical to the operation of the Data Access Control System for content-based sources. Subject-based sources do not require locks. The DACSLock contains the requirements for the requested item. The data flow of a DACSLock and its operation are depicted in the remaining sections of this chapter.

3.2 Forming a DACSLock

The source server is responsible for creating a DACSLock. What information is encoded within the DACSLock is vendor-specific. Two examples are presented to clarify the formation of DACSLocks with respect to a source server. Figure 1 demonstrates the input requirements, the DACSLock API to be called, and the transport mechanism of the DACSLock from the source server to the market data client application.

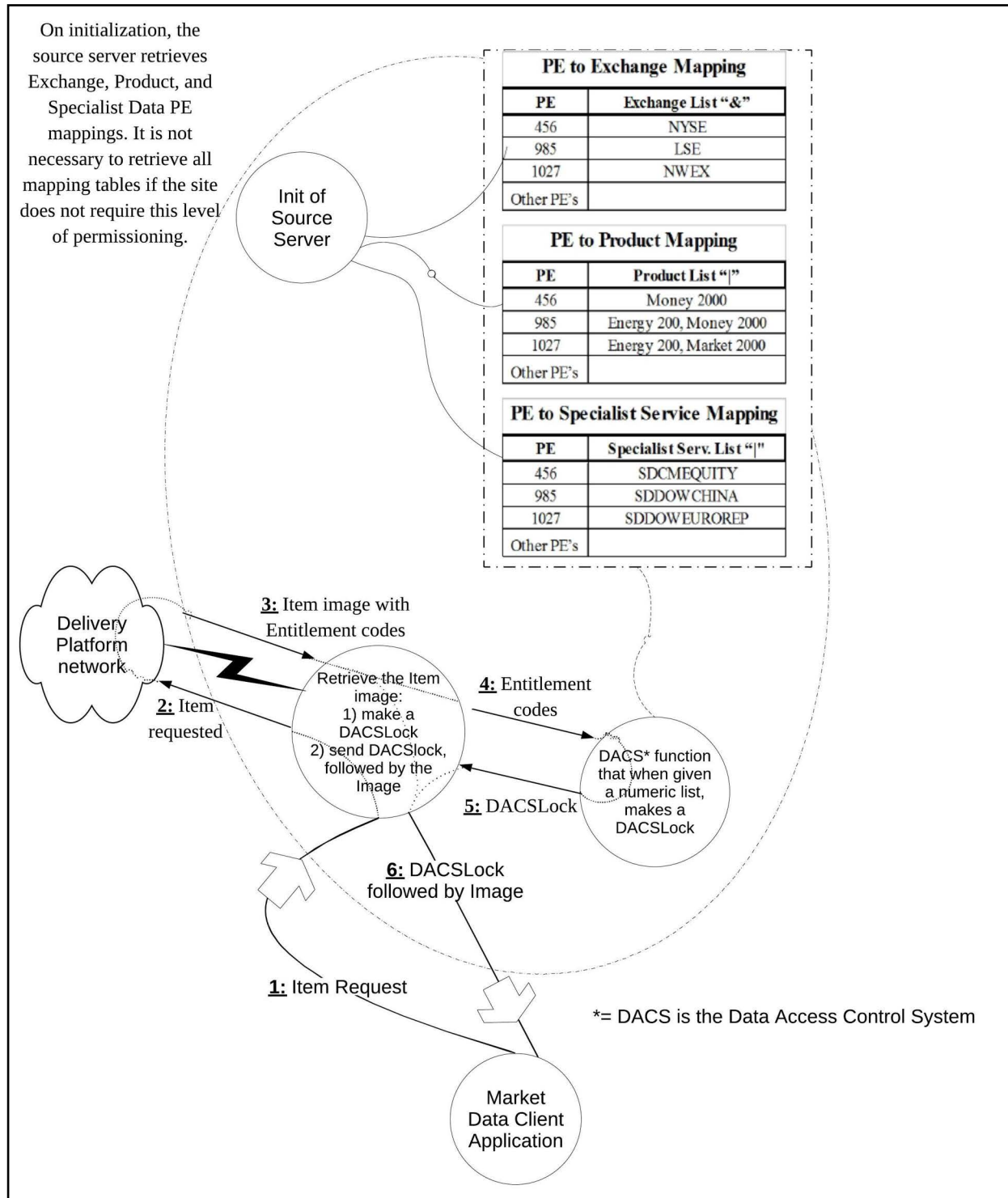


Figure 1. Forming a DACSLock for Real Time Direct

Figure 1 presents the following information:

- The Data Access Control System Station (via use of its map collect utility) is responsible for retrieving Exchange, Product, and Subservice mappings. For dynamic mapping, the Real-Time server must retrieve mapping tables from the datafeed line.
- It is up to the Real-Time server if it retrieves all the mapping tables based on the customer site requirements and on the capabilities of the server's datafeed line.
- The DACSLock API function that makes the DACSLock receives the item's list of entitlement codes. For most Items, this is a single PE. However, other items (e.g., a NEWS2000 Item) can have PE lists that include up to 256 entitlement codes. For this reason, the DACSLock API function might include an operator (**AND/OR**) with its PE list. Thus, in the case of a NEWS2000 item, the PE list is assigned an **OR** operator, while other PE lists might include the **AND** operator.
- The DACSLock must be sent before the image so that the Market Data Client application (via the Transport API) can permission the item as soon as the image arrives, instead of having to hold the image and wait for the Lock.

3.3 DACSLock Contents

A DACSLock must contain information relevant to the requirements for the requested item. Because a DACSLock needs to minimize communication costs, a source server encodes these requirements into a single numeric number. In Figure 1, you can see that the source server creates a table that maps these textual requirements to associated numeric values. By supplying an operator to the DACSLock API function that creates DACSLocks, complex requirements can be managed by the source server. Thus an item's requirements are determined by comparing its PE to the mapping tables.

Figure 1 illustrates the following requirements in the source server mapping tables:

PE	REQUIREMENTS
456	NYSE & Money 2000 & (NY LONDON)
985	LSE & (Energy 2000 Money 2000) & NY
1027	NWEX & (Energy 2000 Markets 2000) & LONDON

Table 4: Item Requirement Formulations

A source server can add requirements to the DACSLock for an item by including extra entitlement codes in the PE list assigned to the appropriate DACSLock API class. For example, if a Source Server specifies that only a **Page_Call** application can use an item, then the source server creates a new PE with that requirement and attaches it to the PE for that item using the AND operator.

PE	REQUIREMENTS
5000	Page_Call

Table 5: PE Example that can Add Extra Requirements to an Item

So if an Item has a **PE = 456**, and the Source Server requires only **Page_Call** access, then the Source Server passes entitlement codes **456** and **5000** as parameters to the appropriate DACSLock API function that builds the DACSLocks from the PE lists.

3.4 Compression of DACSLocks

To minimize physical size, DACSLocks are compressed based on the Binary Coded Decimal (BCD) algorithm. For example, the DACSLock API must make a lock with the following PE requirements:

REAL-TIME (IDN)	BRIDGE
1027 & (456 985) & 5000	1 & 2

Table 6: PE Requirements for a Compound Item

The BCD compression algorithm converts numeric PE values and interprets operator functions according to the following table:

OPERATION	REPLACED NIBBLE VALUE
numeric 0 – 9	0 - 9
"&" and operation	A
" " or operation	B
"EOF" End of DACSLock	C
"EOS" End of Source Server PE List	D

Table 7: Operator and BCD Interpretation Table

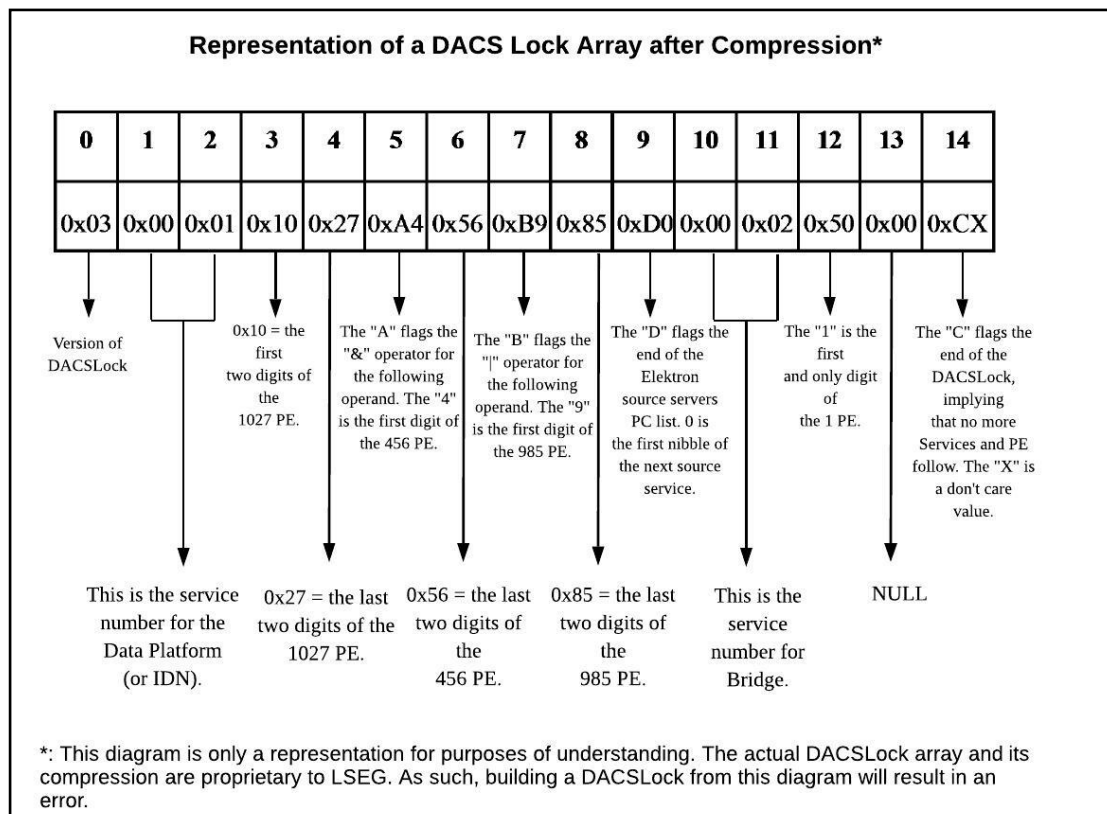


Figure 2. DACSLock Array after Compression

By using the compression mapping in Table 7, the DACSLock API creates the unsigned character array in Figure 2, which illustrates some of the advantages to using this compression:

- The compression does not limit the maximum value of a PE, other than a possible machine maximum that can be manipulated easily with the software language (i.e., $(2^{32})-1$ for an unsigned long).
- BCD compression is simple and fast without resulting in excessive computation.
- The compressed DACSLock is smaller than the actual ASCII string if it were sent.

3.5 Compounding DACSLocks

The previous example, in Figure 2, had two source server PE lists within the DACSLock. This situation is possible when a compound server combines items from more than one source server type. To create a compound DACSLock, the compound server calls the DACSLock API function that, when given the constituent item DACSLocks, creates a DACSLock that contains all of the constituent item requirements. To minimize the size of a DACSLock, the DACSLock API uses Boolean algebra rules to minimize entitlement codes within the PE list whenever possible.

Some rules are:

```
(A | B) & A = A
A & A & B = A & B
A | A | B = A | B
```

A compound server stores all constituent item DACSLocks until the item is no longer required. This functionality is required in the event a new DACSLock is received by the compound server for an open item, in which case the compound server must update the compound item's DACSLock. After updating a DACSLock, the compound server must forward its new compound item DACSLock to those servers that have the compound item open.

3.6 Transport of DACSLocks

DACSLocks for compound servers might grow larger than the maximum size of a single message, in which case the server splits the DACSLock across multiple messages.

3.7 Compound DACSLock in Relation to Permissioning

The previous sections explain the rules for constructing and transporting the DACSLocks. Figure 3 shows the data flow functionality of a DACSLock in an operating environment with two source servers (Bridge and Real Time Direct), a compound server, and a Market Data client application. In this example the Market Data client application requests an item from the compound server. The item is made from constituent items from the Real Time Direct and Bridge Servers.

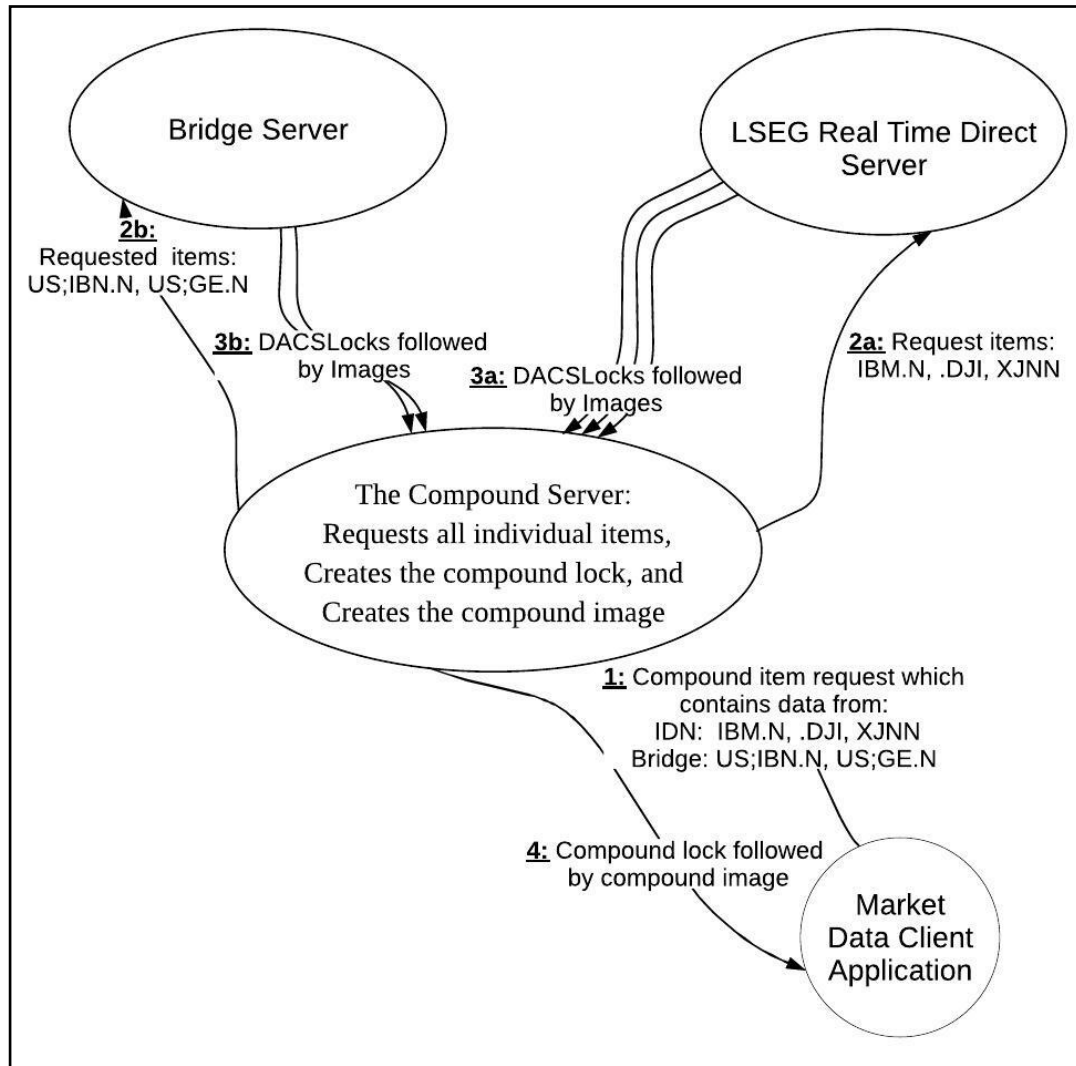


Figure 3. Compound Locks

4 DACSLock API Interface

This chapter describes the DACSLock API components with their required properties. These components are the following:

- AuthorizationLock Interface
- AuthorizationLockData Interface
- AuthorizationLockStatus Interface

4.1 AuthorizationLock Interface

The **AuthorizationLock** interface stores the authorization lock information (i.e. service, operator, and PE list). It is used to supply/modify the authorization lock information and to create the actual DACS lock, wrapped in the **AuthorizationLockData** class.

4.1.1 AuthorizationLock Enumeration

4.1.1.1 OperatorEnum

The **OperatorEnum** enumeration is used to define the operator information of the authorization lock. The operator is the user-supplied operation that shall be imposed on the PE list.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.OperatorEnum
```

Enumeration values:

- **OR**: The character `]`. Specifies that for the user to be allowed access to the protected information, the user must have access to at least one of the PEs contained in the actual DACS lock.
- **AND**: The character `&`. Specifies that for the user to be allowed access to the protected information, the user must have access to all of the PEs contained in the actual DACSLOCK.

4.1.2 AuthorizationLock Construction

4.1.2.1 Constructor

You can use two overloaded **AuthorizationLock** constructors to create the **AuthorizationLock** object.

AuthorizationLock()

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock()
```

Return Value:

Type: **AuthorizationLock**

This **AuthorizationLock** constructor returns an **AuthorizationLock** object.

AuthorizationLock(int, int)

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock(int serviceID, int lockOperator)
```

Parameters:

- **serviceID**: Specifies the service ID in the authorization lock.
- **lockOperator**: Specifies the user-supplied operation that shall be imposed on the PE list.

Return Value:

Type: **AuthorizationLock**

This **AuthorizationLock** constructor returns an **AuthorizationLock** object without PE supplied.

NOTE: You can use the function **AppendPE ()** to supply PE to the authorization lock.

AuthorizationLock(int, int, List<uint>)

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock(int serviceID, int lockOperator, List<uint> peList)
```

Parameters:

- **serviced**: Specifies the service ID in the authorization lock.
- **lockOperator**: Specifies the user-supplied operation that shall be imposed on the PE list.
- **peList**: Specifies the PE list in the authorization lock.

Return Value:

Type: **AuthorizationLock**

This **AuthorizationLock** constructor returns an **AuthorizationLock** object, with supplied service ID, operator and PE list

NOTE: The PE list can be removed by the use of function **RemoveAllPEs ()**. A PE can be appended to the PE list by use of the function **AppendPE ()**.

4.1.2.2 Copy Constructor

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock(AuthorizationLock authorizationLock)
```

Parameters:

- **authorizationLock**: Specifies the **AuthorizationLock** to copy.

Return Value:

Type: **AuthorizationLock**

The copy constructor returns an **AuthorizationLock** object with the same data of the supplied **AuthorizationLock**.

4.1.3 AuthorizationLock Member Functions / Properties

4.1.3.1 AppendPE()

The **AppendPE()** method is used to add the supplied PE to the authorization lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.AppendPE(uint pe)
```

Parameters:

- **pe**: Specifies the PE value to append to the authorization lock.

4.1.3.2 GetLock()

The **GetLock()** method is used to create the actual DACSLOCK.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.GetLock(AuthorizationLockData retLockData, AuthorizationLockStatus retStatus)
```

Parameters:

- **retLockData**: Stores the actual DACS lock created if the return value is **LockResultEnum.LOCK_SUCCESS**. It will not be modified by the **GetLock()** function if the return value is **LockResultEnum.LOCK_FAILURE**.
- **retStatus**: Stores the successful text if the return value is **LockResultEnum.LOCK_SUCCESS**. Stores the detailed failure reason if the return value is **LockResultEnum.LOCK_FAILURE**.

Return Value:

Type: **AuthorizationLockData.LockResultEnum**

The **GetLock()** method returns **LockResultEnum.LOCK_SUCCESS** if the function did not encounter a fatal error. The created DACS lock is stored in **retLockData**. **LockResultEnum.LOCK_FAILURE** is returned if a fatal unrecoverable error was encountered. An explanation of the error can be determined by the **AuthorizationLockStatus.StatusText** property.

NOTE: Constructor **AuthorizationLockData()** may be used to create an empty **AuthorizationLockData** object. Then it is passed into function **AuthorizationLock.GetLock()**.

4.1.3.3 RemoveAllPEs()

The **RemoveAllPEs()** method is used to clear the PE list in the authorization lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.RemoveAllPEs()
```

4.1.3.4 Operator property

The **Operator** property is used to get the operator value.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.Operator { get; }
```

Property Value:

Type: int

The int value which represents the operator value.

4.1.3.5 PeList property

The **PeList** property is used to get the PE list within the lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.PeList { get; }
```

Property Value:

Type: **System.Collections.Generic.List<uint>**

The **List** which represents the PE list within the lock.

4.1.3.6 ServiceID property

The **ServiceID** property is used to get the service ID.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLock.ServiceID { get; }
```

Property Value:

Type: int

The int value which represents the service ID.

4.2 AuthorizationLockData Interface

The **AuthorizationLockData** interface represents the lock data in the DACS lock format. It includes a byte array that references to the DACS lock that represents the PE list in the DACS lock format, and an integer that reflects the DACS lock length. The DACS lock byte array may be retrieved by the use of **LockData** property. The DACS lock length may be retrieved by the use of **Size** property.

The DACS lock operations, which are changing service ID in a simple DACS lock, combining DACS locks into a single composite DACS lock, comparing two DACS locks are provided through this interface.

4.2.1 AuthorizationLockData Enumeration

4.2.1.1 LockResultEnum

The **LockResultEnum** enumeration is used to define the return values of the lock operations in **AuthorizationLock** and **AuthorizationLockData** interfaces.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.LockResultEnum
```

Enumeration values:

- **LOCK_SUCCESS**: Specifies that the operation is successful.
- **LOCK_IDENTICAL**: The result of DACS locks' comparison. Specifies that two DACS locks are identical.
- **LOCK_DIFFERENT**: The result of DACS locks' comparison. Specifies that two DACS locks are different.
- **LOCK_FAILURE**: Specifies that the operation has failed.

4.2.2 AuthorizationLockData Construction

4.2.2.1 Constructor

You can use two overloaded **AuthorizationLockData** constructors to create the **AuthorizationLockData** object.

AuthorizationLockData()

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData()
```

Return Value:

Type: **AuthorizationLockData**

This **AuthorizationLockData** constructor returns an empty **AuthorizationLockData** object.

AuthorizationLockData(byte[], int)

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData(byte[] lockInfo, int lockLen)
```

Parameters:

- **lockInfo**: Specifies an array of byte that references to the lock that represents the PE in DACS lock format.
- **lockLen**: Specifies the length of **lockPtr**.

Return Value:

Type: **AuthorizationLockData**

This **AuthorizationLockData** constructor returns an **AuthorizationLockData** object, with supplied DACS lock information and DACS lock length.

4.2.2.2 Copy Constructor

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData (AuthorizationLockData authorizationLockData)
```

Parameters:

- **authorizationLockData**: Specifies the **AuthorizationLockData** to copy.

Return Value:

Type: **AuthorizationLockData**

The copy constructor returns an **AuthorizationLockData** object with the same PE list in DACS lock format and DACS lock length of the supplied **authorizationLockData**.

4.2.3 AuthorizationLockData Member Functions / Properties

4.2.3.1 ChangeLock()

The **ChangeLock ()** method is used to modify the service ID in the DACS lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.ChangeLock(int oldServiceID, int newServiceID,  
AuthorizationLockStatus retStatus)
```

Parameters:

- **oldServiceID**: Specifies the service ID to be modified in the DACS lock.
- **newServiceID**: Specified the new service ID.
- **retStatus**: Stores the successful text if the return value is **LockResultEnum.LOCK_SUCCESS**. Stores the detailed failure reason if the return value is **LockResultEnum.LOCK_FAILURE**.

Return Value:

Type: **AuthorizationLockData.LockResultEnum**

The **ChangeLock()** method returns **LockResultEnum.LOCK_SUCCESS** if **oldServiceID** is found in the DACS lock and replaced with **newServiceID**. **LockResultEnum.LOCK_FAILURE** is returned if the DACS lock is a compound DACS lock, or **iOldServiceID** is not found in the DACS lock. An explanation of the error can be determined by the **AuthorizationLockStatus.StatusText** property.

NOTE: Using the **ChangeLock ()** function to change service ID in a compound DACS lock will get an operation failure result.

4.2.3.2 CombineLock()

The **CombineLock ()** method is used to combine a list of DACS locks to a single composite DACS lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.CombineLock(List<AuthorizationLockData> lockDataList,  
AuthorizationLockStatus retStatus)
```

Parameters:

- **lockDataList**: Specifies the list of DACS locks to be combined with.
- **retStatus**: Stores the successful text if the return value is `LockResultEnum.LOCK_SUCCESS`. Stores the detailed failure reason if the return value is `LockResultEnum.LOCK_FAILURE`.

Return Value:

Type: `AuthorizationLockData.LockResultEnum`

The `CombineLock()` method returns `LockResultEnum.LOCK_SUCCESS` if the function successfully combined with all DACS locks in `lockDataList` into a single composite DACS lock. After the function `CombineLock()` is called, the DACS lock object stores the single composite DACS lock. `LockResultEnum.LOCK_FAILURE` is returned if a fatal unrecoverable error was encountered. An explanation of the error can be determined by the `AuthorizationLockStatus.StatusText` property.

4.2.3.3 CompareLock()

The `CompareLock()` method is used to compare two DACS locks.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.CompareLock(AuthorizationLockData lockData,
    AuthorizationLockStatus retStatus)
```

Parameters:

- **lockData**: Specifies the DACS lock to be compared with.
- **retStatus**: Stores the successful text if the return value is `AuthorizationLockData.LockResultEnum.LOCK_SUCCESS`. Stores the detailed failure reason if the return value is `AuthorizationLockData.LockResultEnum.LOCK_FAILURE`.

Return Value:

Type: `AuthorizationLockData.LockResultEnum`

The `CompareLock()` method returns `LockResultEnum.LOCK_IDENTICAL` if two locks are logical identical. `LockResultEnum.LOCK_DIFFERENT` is returned if two locks are logical different. `LockResultEnum.LOCK_FAILURE` is returned if a fatal unrecoverable error was encountered. An explanation of the error can be determined by the `AuthorizationLockStatus.StatusText` property.

4.2.3.4 EnumLockData()

The `EnumLockData()` method populates a supplied `System.Collections.Generic.List` of `AuthorizationLock` types to which the `AuthorizationLockData` corresponds. This is useful when converting from a DACSLOCK to a SSL Permission Structure.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.EnumLockData(List<AuthorizationLock> authorizationLockList,
    AuthorizationLockStatus retStatus)
```

Parameters:

- **authorizationLockList**: Specifies an empty `System.Collections.Generic.List` of `AuthorizationLock` types.
- **retStatus**: Stores the successful text if the return value is `AuthorizationLockData.LockResultEnum.LOCK_SUCCESS`. Stores the detailed failure reason if the return value is `AuthorizationLockData.LockResultEnum.LOCK_FAILURE`.

Return Value

Type: `AuthorizationLockData.LockResultEnum`

The `EnumLockData()` method return `AuthorizationLockData.LockResultEnum.LOCK_SUCCESS` if successfully populated the data. Otherwise returns `AuthorizationLockData.LockResultEnum.LOCK_FAILURE`.

4.2.3.5 LockData property

The **LockData** property is used to get the array of byte that references to the DACS lock.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.LockData { get; }
```

Property Value:

Type: byte[]

The byte[] represents an array of byte that references to the DACS lock.

4.2.3.6 Size property

The **Size** property is used to get the DACS lock length.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockData.Size { get; }
```

Property Value:

Type: int

The int value which represents the DACS lock length.

4.3 AuthorizationLockStatus Interface

The **AuthorizationLockStatus** interface represents the status after some operations in **AuthorizationLock** interface and **AuthorizationLockData** interface are performed. The status text is stored as an ASCII string and a Unicode string.

4.3.1 AuthorizationLockStatus Construction

4.3.1.1 Constructor

The **AuthorizationLockStatus()** constructor is used to create an **AuthorizationLockStatus** object.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockStatus()
```

Return Value:

Type: **AuthorizationLockStatus**

The **AuthorizationLockStatus** constructor returns an empty **AuthorizationLockStatus** object.

4.3.1.2 Copy Constructor

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockStatus(AuthorizationLockStatus authorizationLockStatus)
```

Parameters:

- **authorizationLockStatus**: Specifies the **AuthorizationLockStatus** to be copied.

Return Value:

Type: **AuthorizationLockStatus**

The copy constructor returns an **AuthorizationLockStatus** object with the same status text as that of the supplied **AuthorizationLockStatus authorizationLockStatus**.

4.3.2 AuthorizationLockStatus Member Property

4.3.2.1 StatusText property

The **StatusText** property is used to get or set the status text.

Fully Qualified Name:

```
LSEG.Eta.Dacs.AuthorizationLockStatus.StatusText { get; set; }
```

Property Value:

Type: **string**

string represents the status text.

Appendix A Example Program

The following is an example code created using the DACSLock API:

```
/*|-----  
*|          This source code is provided under the Apache 2.0 license      --  
*| and is provided AS IS with no warranty or guarantee of fit for purpose. --  
*|          See the project's LICENSE.md for details.                      --  
*|          Copyright (C) 2024 LSEG. All rights reserved.                  --  
*|-----  
*/  
  
using LSEG.Eta.Dacs;  
  
namespace LSEG.Eta.Example.AuthLock;  
  
public class Program  
{  
    private string lastMethodString = string.Empty;  
    private readonly List<uint> PEList;  
    private readonly AuthorizationLockStatus retStatus;  
  
    private static void Main()  
    {  
        Program program = new();  
        program.PEList.Add(62);  
        AuthorizationLock authLock = new(5000, AuthorizationLock.OperatorEnum.OR, program.PEList);  
  
        Console.WriteLine("\nCreate lockData");  
        AuthorizationLockData lockData = new();  
  
        program.MyGetLock(authLock, lockData);  
  
        Console.WriteLine("\nCreate lockData1");  
        AuthorizationLockData lockData1 = new();  
        authLock.AppendPE(144);  
        program.MyGetLock(authLock, lockData1);  
  
        Console.WriteLine("\nCreate lockData2");  
        AuthorizationLockData lockData2 = new();  
        authLock.AppendPE(62);  
        program.MyGetLock(authLock, lockData2);  
  
        Console.WriteLine("\nCreate lockData3");  
        AuthorizationLockData lockData3 = new();  
        authLock.RemoveAllPEs();  
        program.MyGetLock(authLock, lockData3);  
  
        Console.WriteLine("\nCreate invalid lockData4");  
    }  
}
```

```

int invalidLockLen = lockData2.Size - 1;
byte[] invalidLock = new byte[invalidLockLen];

for (int i = 0; i < invalidLockLen; i++)
{
    invalidLock[i] = lockData2.LockData[i];
}

AuthorizationLockData lockData4 = new(invalidLock, invalidLockLen);

Console.WriteLine("\nCombine lockData, lockData1, and lockData2");
AuthorizationLockData outLockData = new();

Console.WriteLine("\nCombine a different lockData");
AuthorizationLockData outLockData1 = new();
List<AuthorizationLockData> lockDataList = new()
{
    lockData,
    lockData1,
    lockData3
};
program.MyCombineLock(lockDataList, outLockData1);
program.MyCompareLock(outLockData, outLockData1);           // identical

Console.WriteLine("\nAn invalid lock in the combined lock");
AuthorizationLockData outLockData2 = new();
lockDataList.Add(lockData4);
program.MyCombineLock(lockDataList, outLockData2);           // Failure
}

private Program()
{
    PEList = new List<uint>();
    retStatus = new AuthorizationLockStatus();
}

private void MyGetLock(AuthorizationLock authLock,
    AuthorizationLockData retLockData)
{
    lastMethodString = "AuthorizationLock.GetLock()";
    LockResultEnum result = authLock.GetLock(retLockData, retStatus);

    if (result == LockResultEnum.LOCK_SUCCESS)
    {
        Console.WriteLine(lastMethodString.ToString() + " - Success ");
    }
    else
    {
        Console.WriteLine(lastMethodString.ToString() + " - Failure: " +
retStatus.StatusText.ToString());
    }
}

```

```

}

private void MyCompareLock(AuthorizationLockData lockData1,
AuthorizationLockData lockData2)
{
    lastMethodString = "AuthorizationLockData.CompareLock()";
    LockResultEnum result = lockData1.CompareLock(lockData2, retStatus);

    if (result == LockResultEnum.LOCK_IDENTICAL)
    {
        Console.WriteLine(lastMethodString.ToString() + " - Two locks are identical");
    }
    else if (result == LockResultEnum.LOCK_DIFFERENT)
    {
        Console.WriteLine(lastMethodString.ToString() + " - Two locks are different.");
    }
    else
    {
        Console.WriteLine(lastMethodString.ToString() + " - Failure: " +
retStatus.StatusText.ToString());
    }
}

private void MyCombineLock(List<AuthorizationLockData> lockDataList,
AuthorizationLockData retLockData)
{
    lastMethodString = "AuthorizationLockData.CombineLock()";
    LockResultEnum result = retLockData.CombineLock(lockDataList, retStatus);

    if (result == LockResultEnum.LOCK_SUCCESS)
    {
        Console.WriteLine(lastMethodString.ToString() + " - Success");
    }
    else
    {
        Console.WriteLine(lastMethodString.ToString() + " - Failure: " +
retStatus.StatusText.ToString());
    }
}
}

```

© LSEG 2024 - 2026. All rights reserved.

Republication or redistribution of LSEG Data & Analytics content, including by framing or similar means, is prohibited without the prior written consent of LSEG Data & Analytics. 'LSEG Data & Analytics' and the LSEG Data & Analytics logo are registered trademarks and trademarks of LSEG Data & Analytics.

Any third party names or marks are the trademarks or registered trademarks of the relevant third party.

Document ID: ETACSharp351L1EDAC.260
Date of issue: March 2026



LSEG DATA &
ANALYTICS